

SECURITY REVIEW

для Claude Code

Полный гайд по аудиту безопасности вашего проекта.
7 промптов + /security-review + GitHub Action.
Для VS Code, Cursor, Windows, Mac, Linux.

Этот гайд объединяет три подхода к security review:

1. Встроенная команда /security-review в Claude Code
2. 7 целевых промптов для проверки конкретных аспектов
3. GitHub Action для автоматической проверки каждого PR

Каждый промпт — copy-paste ready. Вставляете в чат Claude Code, получаете детальный отчёт с конкретными исправлениями.

v1.0 · 2026

Гайд подготовлен @alex_magnier

Содержание

- 01 Зачем нужен Security Review
- 02 Быстрый старт: /security-review
- 03 Промпт 1 — Аутентификация
- 04 Промпт 2 — Авторизация и IDOR
- 05 Промпт 3 — Валидация ввода
- 06 Промпт 4 — Секреты и API-ключи
- 07 Промпт 5 — Rate Limiting и защита от ботов
- 08 Промпт 6 — Деплой и мониторинг
- 09 Промпт 7 — Полный аудит безопасности
- 10 GitHub Action: автопроверка PR
- 11 Кастомная команда для проекта
- 12 Чеклист готовности

01 — Зачем нужен Security Review

AI-генерированный код ускоряет разработку в разы, но создаёт новый класс рисков. Исследования показывают: код, написанный с помощью AI, в 2—3 раза чаще содержит уязвимости вроде XSS и небезопасных ссылок на объекты. Claude Code мощный — но без проверки безопасности вы отправляете в продакшн код, который никто не ревьюил.

Без Security Review	С Security Review
RLS-политики с дырами	RLS протестирована для каждой роли
API-ключи в клиентском коде	Секреты только в env-переменных
SQL/XSS через формы	Zod-валидация на каждом входе
Нет rate limiting — боты выедают бюджет	Rate limiting на auth и AI-эндпоинтах
Один пользователь видит данные другого	Ownership check на каждом запросе
Обнаружение — когда уже взломали	Логирование ловит подозрительное заранее

В феврале 2026 Anthropic запустил Claude Code Security — enterprise-продукт, который на модели Opus 4.6 нашёл более 500 уязвимостей в open-source проектах, пропущенных экспертами за десятилетия. Те же возможности доступны вам бесплатно через /security-review и промпты из этого гайда.

Три уровня защиты

/security-review — общий скан всего проекта (быстро, широко)

7 целевых промптов — глубокая проверка конкретных аспектов (точно, детально)

GitHub Action — автоматическая проверка каждого PR (непрерывно, без участия)

02 — Быстрый старт: /security-review

Claude Code имеет встроенную команду для security review. Она доступна всем пользователям на любом плане (Pro, Max, Team, Enterprise). Это самый быстрый способ проверить проект — одна команда, полный отчёт.

Как использовать

1. Откройте проект в VS Code с установленным Claude Code
2. Откройте чат Claude Code (боковая панель)
3. Введите /security-review и нажмите Enter
4. Claude проанализирует кодовую базу и выдаст отчёт
5. Для каждой найденной уязвимости попросите Claude исправить её

Что проверяет

Команда использует специализированный промпт, который ищет распространённые паттерны уязвимостей:

- SQL-инъекции и небезопасные запросы к базе данных
- XSS (Cross-Site Scripting) уязвимости
- Проблемы аутентификации и авторизации
- Небезопасная обработка данных
- Уязвимости зависимостей (dependency vulnerabilities)

Важно: /security-review — это широкий скан. Он находит очевидные проблемы, но может пропустить специфичные уязвимости вашего стека (например, дырявые RLS-политики Supabase или утечку токенов в Telegram-боте). Для глубокой проверки используйте целевые промпты из следующих разделов.

Когда запускать

Рекомендация: запускайте /security-review перед каждым значительным коммитом. В Anthropic этой командой проверяют собственный код — и она уже нашла RCE-уязвимость через DNS rebinding и SSRF в прокси-системе до того, как они попали в продакшн.

03 — Промпт 1 — Аутентификация

Безопасность аутентификации

Аутентификация — первая линия обороны. Если она дырявая, всё остальное не имеет смысла. Этот промпт проверяет: как хранятся пароли, как работают сессии, защищён ли вход от брутфорса, не утекают ли токены на фронтенд.

ПРОМПТ (скопируй в чат Claude Code):

Выступи в роли старшего инженера по безопасности. Проведи полный аудит системы аутентификации этого проекта.

Проверь и исправь:

1. Пароли хешируются надёжно (bcrypt/argon2, не md5/sha1)
2. Сессии имеют срок жизни и корректно инвалидируются при логауте
3. Email-верификация включена для новых аккаунтов
4. Токены сброса пароля одноразовые и истекают через 1 час
5. Login-эндпоинт имеет rate limiting (макс. 5 попыток в минуту)
6. Auth-секреты (JWT_SECRET, SUPABASE_SERVICE_ROLE_KEY) никогда не попадают в клиентский код
7. OAuth-редиректы валидируются (нет open redirect)
8. Middleware проверяет сессию на защищённых маршрутах

Для каждой найденной проблемы: опиши уязвимость, покажи где в коде, предложи конкретный фикс. Рефакторь небезопасную логику.

Зачем: Предотвращает захват аккаунтов: пароли хранятся безопасно, брутфорс заблокирован rate limiting, сессии не могут быть украдены или переиспользованы после логута.

Supabase-проекты: Supabase Auth закрывает большую часть этих проверок из коробки. Но проверьте: не используете ли вы service_role_key на клиенте, настроены ли email-шаблоны, работает ли middleware в Next.js для проверки сессий.

04 — Промпт 2 — Авторизация и IDOR

Защита от доступа к чужим данным

IDOR (Insecure Direct Object Reference) — одна из самых частых уязвимостей. Пользователь подменяет ID в запросе и получает доступ к чужим данным. Этот промпт проверяет, что каждый запрос к данным содержит проверку владельца.

ПРОМПТ (скопируй в чат Claude Code):

Проверь все API-эндпоинты и запросы к базе данных в этом проекте. Убедись, что каждый запрос проверяет — является ли текущий пользователь владельцем запрашиваемых данных.

Найди и исправь:

1. Эндпоинты, где ID ресурса берётся из URL/body без проверки ownership
2. SELECT-запросы без WHERE user_id = auth.uid()
3. UPDATE/DELETE без проверки принадлежности записи
4. API-роуты, доступные без аутентификации (если должны быть закрыты)
5. Supabase RLS-политики: покрывают ли все таблицы и все операции (SELECT, INSERT, UPDATE, DELETE)
6. Server Actions и Route Handlers: проверяется ли auth.getUser() перед каждой операцией

Для каждой проблемы покажи: какой эндпоинт уязвим, как атакующий может получить чужие данные, конкретный фикс с кодом.

Зачем: Даже если пользователь авторизован — это не значит, что он может видеть ВСЕ данные. IDOR позволяет подменить ?id=123 на ?id=456 и получить чужие заказы, профили, платежи. RLS + ownership checks закрывают это.

Supabase-проекты: RLS — ваш главный щит. Проверьте: 1) RLS включена на КАЖДОЙ таблице (ALTER TABLE ... ENABLE ROW LEVEL SECURITY) 2) Политики покрывают все 4 операции 3) Нет запросов через service_role, которые обходят RLS без необходимости.

05 — Промпт 3 — Валидация ввода

Валидация и санитизация пользовательского ввода

Любое место, где пользователь отправляет данные — форма, API, загрузка файла, query-параметры — это потенциальная точка атаки. SQL-инъекции, XSS, command injection начинаются с невалидированного ввода.

ПРОМПТ (скопируй в чат Claude Code):

Найди каждое место в проекте, где пользовательский ввод попадает в систему: формы, API-эндпоинты, загрузка файлов, query-параметры, URL-параметры.

Для каждой точки входа проверь и исправь:

1. Есть ли Zod-схема (или аналогичная валидация) на сервере
2. SQL-запросы используют параметризованные запросы, не конкатенацию строк
3. HTML-вывод экранирует пользовательский контент (нет dangerouslySetInnerHTML с сырыми данными)
4. Загрузка файлов: проверяется MIME-тип, размер, расширение
5. Redirect URL валидируется (нет open redirect)
6. JSON-поля (jsonb) имеют схему валидации, не принимают произвольную структуру
7. Числовые параметры (limit, offset, page) имеют верхнюю границу

Отклоняй невалидные данные с понятными сообщениями об ошибках. Применяй строгие типы на каждом входе.

Зачем: Останавливает атаки через инъекцию вредоносных команд и кода — через формы, API, загрузку файлов. Без валидации атакующий может выполнить SQL, вставить скрипт в страницу или загрузить исполняемый файл.

06 — Промпт 4 — Секреты и API-ключи

Защита секретов и учётных данных

Утечка одного API-ключа может стоить тысячи долларов — или полный доступ к базе данных. Этот промпт сканирует весь проект на предмет открытых секретов и проверяет, что credentials используются только на сервере.

ПРОМПТ (скопируй в чат Claude Code):

Просканируй весь проект на наличие секретов и учётных данных.

Проверь и исправь:

1. API-ключи, database URLs, service keys не захардкожены в коде
2. .env файлы добавлены в .gitignore
3. .env.example содержит только плейсхолдеры, не реальные значения
4. SUPABASE_SERVICE_ROLE_KEY используется ТОЛЬКО в серверном коде (API Routes, Server Actions, Edge Functions)
5. NEXT_PUBLIC_* переменные не содержат секреты (всё с этим префиксом видно в браузере)
6. ЮKassa secret_key и Stripe secret key не попадают на клиент (на клиенте только shopId / publishable key)
7. ANTHROPIC_API_KEY используется только на сервере
8. Git-история не содержит ранее закоммиченных секретов
9. Токены аутентификации хранятся в httpOnly cookies, не в localStorage

Перенеси все секреты в переменные окружения. Убедись что они используются только на сервере.

Зачем: API-ключи и credentials в клиентском коде — прямой путь к взлому. Атакующий открывает DevTools, находит ключ и получает полный доступ к вашей базе данных, AI-сервисам или платёжной системе.

Критично для Supabase: SUPABASE_SERVICE_ROLE_KEY обходит BCE RLS-политики. Если он попал на клиент — любой пользователь имеет полный доступ к БД. Используйте только NEXT_PUBLIC_SUPABASE_ANON_KEY на клиенте.

07 — Промпт 5 — Rate Limiting и боты

Rate Limiting и защита от злоупотреблений

Без rate limiting один бот может: подбирать пароли к аккаунтам, исчерпать лимит вашего Anthropic API за минуты, скрейпить все данные из каталога, создать тысячи фейковых аккаунтов. Этот промпт закрывает все эти вектора.

ПРОМПТ (скопируй в чат Claude Code):

Добавь защиту от злоупотреблений в приложение.

Реализуй и проверь:

1. Rate limiting на login-эндпоинте: макс. 5 попыток / минуту на IP
2. Rate limiting на регистрации: макс. 3 аккаунта / час на IP
3. Rate limiting на AI-генерации: по плану подписки (free: 5/день, pro: 50/день)
4. Rate limiting на API-эндпоинтах: макс. 100 запросов / минуту на пользователя
5. Защита от перебора ID: не используй последовательные числовые ID (используй UUID)
6. CAPTCHA или proof-of-work на публичных формах (регистрация, контакт)
7. Защита вебхуков: проверка подписи (ЮKassa SHA-256 HMAC, Stripe signature, Telegram secret token)
8. Лимиты на размер запроса (body size limit: 1MB для API, 10MB для загрузок)

Для реализации: Upstash Redis + @upstash/ratelimit для serverless, или middleware на базе in-memory Map для MVP.

Зачем: Rate limiting останавливает ботов от перебора паролей, скрейпинга данных, массовой отправки запросов которые могут обрушить приложение или исчерпать бюджет на AI API.

08 — Промпт 6 — Деплой и мониторинг

Безопасность деплоя и мониторинг

Код может быть безупречен, но если деплой настроен неправильно — база данных открыта из интернета, логи не пишутся, HTTPS не включён — всё без толку. Этот промпт проверяет инфраструктурную безопасность.

ПРОМПТ (скопируй в чат Claude Code):

Настрой безопасный деплой и мониторинг для этого приложения.

Проверь и настрой:

1. HTTPS принудительный (редирект с HTTP, HSTS-заголовок)
2. Секреты хранятся в Vercel Environment Variables / Supabase Vault, не в коде
3. Прямой доступ к базе данных заблокирован из публичного интернета
4. CORS настроен строго: только ваш домен, не wildcard '*'
5. Security-заголовки: X-Frame-Options, X-Content-Type-Options, CSP
6. Логирование: auth-попытки (успешные и неуспешные), API-ошибки (4xx, 5xx), подозрительная активность (много 401/403 с одного IP)
7. Мониторинг аномалий: резкий рост запросов, необычные паттерны
8. Бэкапы БД настроены (Supabase: ежедневные автобэкапы включены)
9. Preview deployments (Vercel) не имеют доступа к продакшн БД

Реализуй структурированное логирование для подозрительных событий.

Зачем: Защищает данные при передаче, предотвращает несанкционированный доступ к системе и помогает обнаружить подозрительное поведение прежде, чем оно станет серьёзной проблемой.

09 — Промпт 7 — Полный аудит

Полный аудит безопасности

Финальный промпт — комплексная проверка всего проекта. Запускайте после всех целевых проверок или как единственный промпт, если нет времени на пошаговый аудит. Покрывает все аспекты за один проход.

ПРОМПТ (скопируй в чат Claude Code):

Проведи полный аудит безопасности этого проекта. Ты — senior security engineer с опытом в Next.js, Supabase, ЮKassa/Stripe и Anthropic API.

Проверь ВСЕ аспекты:

- ☐ Аутентификация: хеширование, сессии, rate limiting на login
- ☐ Авторизация: RLS на каждой таблице, ownership checks в API
- ☐ Ввод: Zod-валидация, защита от SQL/XSS/command injection
- ☐ Секреты: нет ключей в коде, .env в .gitignore, service_role только на сервере
- ☐ Rate limiting: auth, API, AI-генерация, регистрация
- ☐ Деплой: HTTPS, CORS, security headers, логирование
- ☐ Зависимости: npm audit, нет известных CVE в пакетах
- ☐ Edge cases: что если AI API недоступен, что если ЮKassa/Stripe webhook придёт дважды

Формат отчёта для каждой проблемы:

Уровень: КРИТИЧНО / ВАЖНО / РЕКОМЕНДАЦИЯ

Файл: путь/к/файлу.ts:строка

Проблема: что не так

Риск: что может случиться

Фикс: конкретный код исправления

Зачем: Комплексная проверка всех аспектов за один проход. Используйте как финальный чеклист перед деплоем в продакшн или как единственный промпт, если нет времени на пошаговый аудит.

10 — GitHub Action: автопроверка PR

GitHub Action от Anthropic автоматически проверяет каждый Pull Request на уязвимости. Он анализирует только изменённые файлы, оставляет комментарии с найденными проблемами и фильтрует false positives.

Установка

1. Добавьте API-ключ в секреты репозитория:

GitHub → Settings → Secrets → Actions → New secret → CLAUDE_API_KEY

2. Создайте файл `.github/workflows/security-review.yml`:

security-review.yml

```
name: Security Review
permissions:
  pull-requests: write
  contents: read
on:
  pull_request:
  jobs:
    security:
      runs-on: ubuntu-latest
      steps:
        - uses: actions/checkout@v4
          with:
            ref: ${ github.event.pull_request.head.sha }
            fetch-depth: 2
        - uses: anthropics/claude-code-security-review@main
          with:
            comment-pr: true
            claude-api-key: ${ secrets.CLAUDE_API_KEY }
```

Важно: API-ключ должен иметь доступ и к Claude API, и к Claude Code. Action не защищён от prompt injection — используйте только для доверенных PR. Включите «Require approval for all external contributors» в настройках репозитория.

11 — Кастомная команда для проекта

Стандартный `/security-review` — универсальный. Но вы можете создать свою версию, заточенную под ваш стек. Например, если вы работаете с Supabase + ЮKassa/Stripe + Anthropic API — добавьте специфичные проверки.

Как создать

1. Скопируйте `security-review.md` из `github.com/anthropics/claude-code-security-review`
2. Поместите в `.claude/commands/security-review.md` вашего проекта
3. Добавьте проверки специфичные для вашего стека (примеры ниже)
4. Теперь `/security-review` будет использовать вашу версию

Что добавить для Supabase + Next.js + ЮKassa/Stripe стека

- RLS включена и протестирована на КАЖДОЙ таблице
- `service_role_key` не используется в клиентском коде или Server Components без проверки
- ЮKassa: подпись уведомлений проверяется через SHA-256 HMAC от секретного ключа
- Stripe webhook signature проверяется через `constructEvent()` (если используется)
- Anthropic API key не утекает в клиентский бандл
- Edge Functions используют `Deno.env.get()` для секретов
- Middleware проверяет сессию Supabase Auth на каждом защищённом маршруте
- ЮKassa `secret_key` и Stripe `secret key` только в серверных переменных окружения
- `preview deployments` не подключены к продакшн Supabase

Совет: Добавьте этот файл в репозиторий и закоммитьте. Все члены команды получат одинаковую кастомизированную проверку. При обновлении стека — обновляйте и проверки.

12 — Чеклист готовности

Перед деплоем в продакшн пройдите этот чеклист. Каждый пропущенный пункт — потенциальная уязвимость.

Аутентификация

- ☐ Пароли хешируются (bcrypt/argon2)
- ☐ Сессии имеют TTL и инвалидируются при логауте
- ☐ Rate limiting на login (5 попыток/мин)
- ☐ Email-верификация включена
- ☐ OAuth-редиректы валидируются

Авторизация

- ☐ RLS включена на каждой таблице
- ☐ RLS покрывает SELECT, INSERT, UPDATE, DELETE
- ☐ Ownership check в каждом API-эндпоинте
- ☐ auth.getUser() вызывается на сервере, не доверяем клиенту
- ☐ Нет прямых запросов через service_role без необходимости

Ввод и данные

- ☐ Zod-валидация на каждом серверном эндпоинте
- ☐ Параметризованные SQL-запросы (нет конкатенации строк)
- ☐ HTML-экранирование пользовательского контента
- ☐ Лимиты на размер загрузок и запросов
- ☐ UUID вместо последовательных ID

Секреты

- ☐ .env в .gitignore
- ☐ .env.example без реальных значений
- ☐ service_role_key только на сервере
- ☐ NEXT_PUBLIC_* не содержит секретов
- ☐ ЮKassa secret_key / Stripe secret key только на сервере
- ☐ Токены в httpOnly cookies, не localStorage
- ☐ Git-история чиста от секретов

Rate Limiting

- ☐ Auth-эндпоинты: 5 попыток/мин
- ☐ API: 100 запросов/мин на пользователя
- ☐ AI-генерация: лимит по плану подписки

- [] Регистрация: 3 аккаунта/час на IP
- [] ЮKassa/Stripe webhook подпись проверяется

Деплой

- [] HTTPS принудительный
- [] CORS: только ваш домен
- [] Security headers настроены
- [] Логирование auth-событий и ошибок
- [] Бэкапы БД включены
- [] Preview deployments изолированы от продакшна

«Security review — не разовое мероприятие, а часть цикла разработки.
/security-review перед каждым коммитом. GitHub Action на каждом PR.
Целевые промпты после каждого модуля.»

Рекомендуемый workflow

Когда	Что делать	Как
После каждого модуля	Целевой промпт по аспекту	Промпты 1—6 из этого гайда
Перед коммитом	Быстрый скан	/security-review
На каждом PR	Автоматическая проверка	GitHub Action
Перед деплоем	Полный аудит	Промпт 7 + чеклист

Полезные ссылки

Claude Code Security Review (GitHub)
github.com/anthropics/claude-code-security-review

Claude Code Docs
docs.claude.com

Anthropic: Claude Code Security
anthropic.com/news/claude-code-security

Гайд подготовлен @alex_magnier
Instagram: @alex_magnier
Telegram: t.me/aidev_alexmagnier

Подписывайтесь — пишу про AI-разработку, Claude Code и автоматизацию.